A **binary heap** is a **complete binary tree** that satisfies the **heap property**:

- **Max-Heap:** Each parent node is **greater than or equal to** its children.
- **Min-Heap:** Each parent node is **less than or equal to** its children.

🔑 Key Points Before Construction:

1. **Complete Binary Tree:**
   - All levels except possibly the last are completely filled.
   - Nodes are filled from **left to right**.
2. **Heap Property:**
   - Max-heap → Largest element is always at the root.
   - Min-heap → Smallest element is always at the root.

🏗️ Construction Methods

There are **two main ways** to build a binary heap from `n` elements.

Method 1: Insertion + Heapify Up

1. Start with an empty tree.
2. Insert elements one by one.
3. After inserting each element, **heapify up** (bubble-up) to maintain heap property.

⏱️ **Time Complexity:**

- Each insertion takes `O(log n)` (heapify up).
- Total time: `O(n log n)`.

Method 2: Heapify (Bottom-Up Construction) – More Efficient ✅

This is the standard and more efficient approach.

Steps:

1. Place all `n` elements in an array in the order they are given.
   (This already forms a complete binary tree because arrays map directly to binary trees.)
2. Start from the **last non-leaf node** and call **heapify-down** on each node moving upward to the root.

🔢 **Index formula for heap (array-based):**

- For node at index `i`
  - **Left child:** `2*i + 1`
  - **Right child:** `2*i + 2`
  - **Parent:** `(i-1) // 2`

Pseudo-code for Bottom-Up Heap Construction:

pseudo Copy code

```
BUILD-HEAP(A):
    n = length(A)
    for i = (n // 2) - 1 downto 0:
        HEAPIFY(A, i, n)
```

Where **HEAPIFY(A, i, n)** ensures the subtree rooted at `i` satisfies heap property by swapping with its largest (or smallest) child and recursively calling `HEAPIFY` if needed.

⏱️ **Time Complexity:** `O(n)` ✅
(This is faster than `O(n log n)` because most nodes are near the bottom and require fewer swaps.)

🖼️ Example (Max-Heap Construction)

Given array:

```csharp
[4, 10, 3, 5, 1]
```

1. Treat array as tree:

```markdown
        4
      /   \
     10     3
   /  \
  5    1
```

2. Start heapify from last non-leaf node → index `1` (value `10`).
   - Left = 5, Right = 1 → 10 is already largest → no change.
3. Heapify index `0` (value `4`):
   - Left = 10, Right = 3 → 10 is largest → swap 4 and 10:

```markdown
        10
      /    \
     4       3
   / \
  5   1
```

4. Heapify index `1` again (value `4`):
    - Left = 5, Right = 1 → 5 is largest → swap 4 and 5:

markdown

```
        10
      /    \
     5       3
    / \
   4   1
```

✅ Final Max-Heap: `[10, 5, 3, 4, 1]`

Heap Sort works in **two phases**:

1. **Build a Max-Heap** from the input array.
2. **Extract the maximum repeatedly** (swap with the last element, shrink heap size, and heapify again).

Since a **max-heap** always has the largest element at the root, we can systematically place the largest elements at the end of the array — resulting in a sorted array.

📝 Heap Sort Algorithm (Pseudo-code)

pseudo

```
HEAPSORT(A):
    n = length(A)

    // Step 1: Build a max heap
    for i = (n//2) - 1 downto 0:
        HEAPIFY(A, i, n)

    // Step 2: Extract elements one by one
    for i = n-1 downto 1:
        swap(A[0], A[i])     // Move current root (max) to end
        HEAPIFY(A, 0, i)     // Heapify reduced heap
```

Where `HEAPIFY(A, i, size)` maintains the max-heap property for subtree rooted at index `i`.

🔎 Example (Heap Sort on `[4, 10, 3, 5, 1]`)

1. **Build Max-Heap:**
   - We already saw that it becomes: `[10, 5, 3, 4, 1]`
2. **Extract Max (10):**
   - Swap with last → `[1, 5, 3, 4, 10]`
   - Heapify reduced array (size=4) → `[5, 4, 3, 1, 10]`
3. **Extract Max (5):**
   - Swap with last in heap → `[1, 4, 3, 5, 10]`
   - Heapify (size=3) → `[4, 1, 3, 5, 10]`
4. **Extract Max (4):**
   - Swap with last in heap → `[3, 1, 4, 5, 10]`
   - Heapify (size=2) → `[3, 1, 4, 5, 10]`
5. **Extract Max (3):**
   - Swap with last → `[1, 3, 4, 5, 10]`
   - Heapify (size=1) — done.

✅ Final Sorted Array:

```csharp
[1, 3, 4, 5, 10]
```

⏱️ Time Complexity

- **Heap Construction:** `O(n)`
- **Repeated Extraction:** `n` extractions × `O(log n)` heapify = `O(n log n)`
- **Total:** `O(n log n)`

✅ Heap Sort is **in-place** (no extra array needed) and **not stable** (relative order of equal elements may change).

Binary Heap

of the data structure. In this section, we will refer to binary heaps merely as *heaps*. Like binary search trees, heaps have two properties, namely, a structure property and a heap-order property. As with AVL trees, an operation on a heap can destroy one of the properties,

6.3.1 Structure Property

A heap is a binary tree that is completely filled, with the possible exception of the bottom level, which is filled from left to right. Such a tree is known as a **complete binary tree**. Figure 6.2 shows an example.

It is easy to show that a complete binary tree of height h has between 2^h and $2^{h+1} - 1$ nodes. This implies that the height of a complete binary tree is $\lfloor \log N \rfloor$, which is clearly $O(\log N)$.

An important observation is that because a complete binary tree is so regular, it can be represented in an array and no links are necessary. The array in Figure 6.3 corresponds to the heap in Figure 6.2.

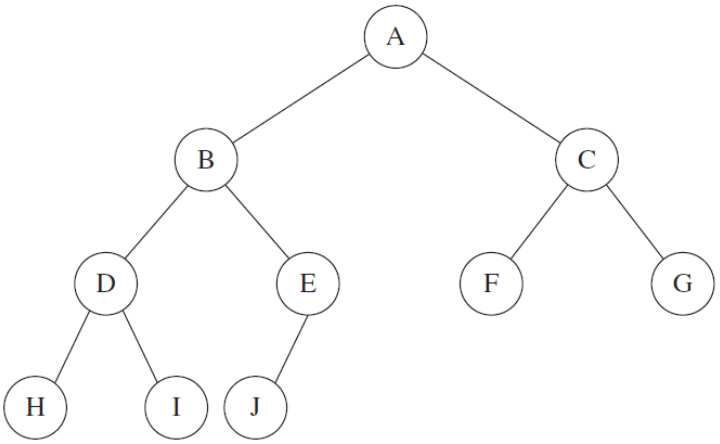

Figure 6.2 A complete binary tree

	A	B	C	D	E	F	G	H	I	J			
0	1	2	3	4	5	6	7	8	9	10	11	12	13

Figure 6.3 Array implementation of complete binary tree

For any element in array position i, the left child is in position $2i$, the right child is in the cell after the left child $(2i + 1)$, and the parent is in position $\lfloor i/2 \rfloor$. Thus, not only are links not required, but the operations required to traverse the tree are extremely simple and likely to be very fast on most computers. The only problem with this implementation is that an estimate of the maximum heap size is required in advance, but typically this is not a problem (and we can resize if needed). In Figure 6.3 the limit on the heap size is 13 elements. The array has a position 0; more on this later.

A heap data structure will, then, consist of an array (of `Comparable` objects) and an integer representing the current heap size. Figure 6.4 shows a priority queue interface.

Throughout this chapter, we shall draw the heaps as trees, with the implication that an actual implementation will use simple arrays.

6.3.2 Heap-Order Property

The property that allows operations to be performed quickly is the **heap-order property**. Since we want to be able to find the minimum quickly, it makes sense that the smallest element should be at the root. If we consider that any subtree should also be a heap, then any node should be smaller than all of its descendants.

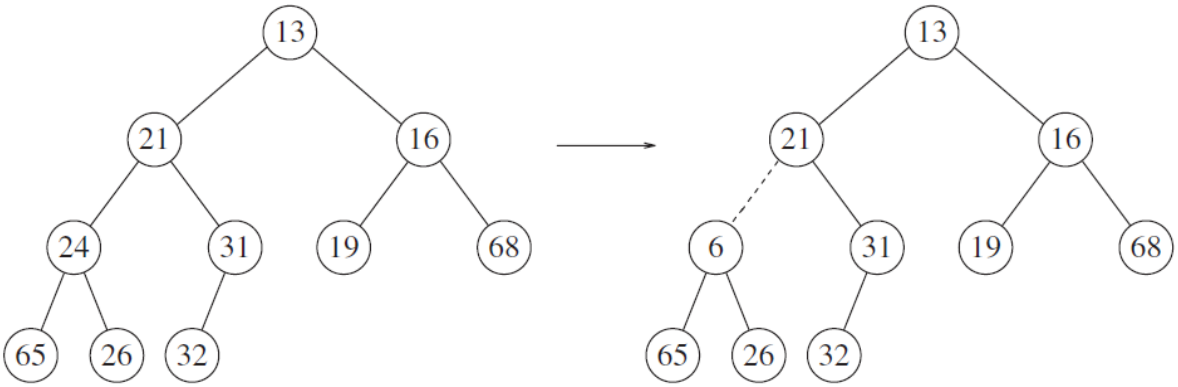

Figure 6.5 Two complete trees (only the left tree is a heap)

Applying this logic, we arrive at the heap-order property. In a heap, for every node X, the key in the parent of X is smaller than (or equal to) the key in X, with the exception of the root (which has no parent).[2] In Figure 6.5 the tree on the left is a heap, but the tree on the right is not (the dashed line shows the violation of heap order).

By the heap-order property, the minimum element can always be found at the root. Thus, we get the extra operation, `findMin`, in constant time.

6.3.3 Basic Heap Operations

It is easy (both conceptually and practically) to perform the two required operations. All the work involves ensuring that the heap-order property is maintained.

`insert`

To insert an element X into the heap, we create a hole in the next available location, since otherwise, the tree will not be complete. If X can be placed in the hole without violating heap order, then we do so and are done. Otherwise, we slide the element that is in the hole's parent node into the hole, thus bubbling the hole up toward the root. We continue this process until X can be placed in the hole. Figure 6.6 shows that to insert 14, we create a hole in the next available heap location. Inserting 14 in the hole would violate the heap-order property, so 31 is slid down into the hole. This strategy is continued in Figure 6.7 until the correct location for 14 is found.

This general strategy is known as a **percolate up**; the new element is percolated up the heap until the correct location is found. Insertion is easily implemented with the code shown in Figure 6.8.

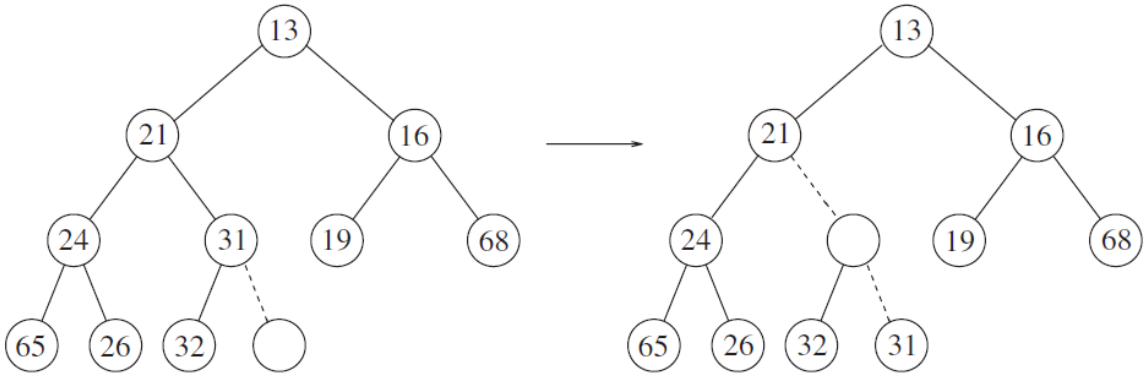

Figure 6.6 Attempt to insert 14: creating the hole, and bubbling the hole up

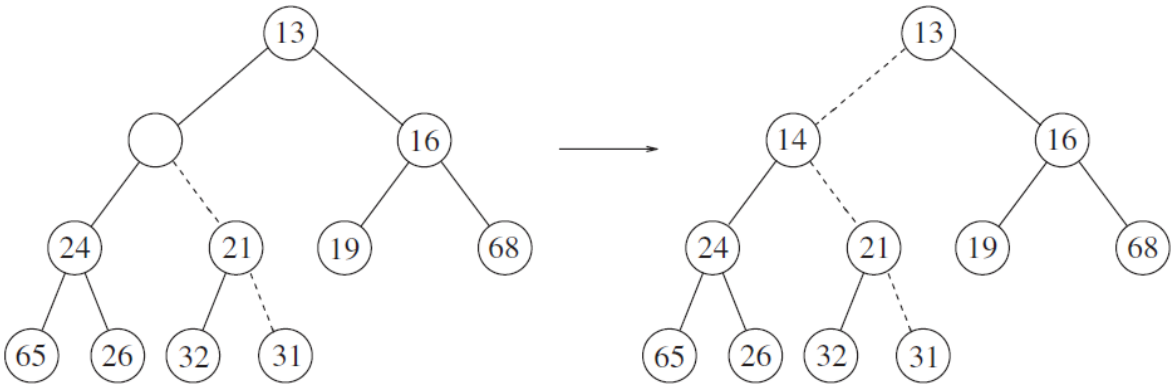

Figure 6.7 The remaining two steps to insert 14 in previous heap

deleteMin

`deleteMin`s are handled in a similar manner as insertions. Finding the minimum is easy; the hard part is removing it. When the minimum is removed, a hole is created at the root. Since the heap now becomes one smaller, it follows that the last element X in the heap must move somewhere in the heap. If X can be placed in the hole, then we are done. This is unlikely, so we slide the smaller of the hole's children into the hole, thus pushing the hole down one level. We repeat this step until X can be placed in the hole. Thus, our action is to place X in its correct spot along a path from the root containing *minimum* children.

In Figure 6.9 the left figure shows a heap prior to the `deleteMin`. After 13 is removed, we must now try to place 31 in the heap. The value 31 cannot be placed in the hole, because this would violate heap order. Thus, we place the smaller child (14) in the hole, sliding the hole down one level (see Fig. 6.10). We repeat this again, and since 31 is larger than 19, we place 19 into the hole and create a new hole one level deeper. We then place 26 in the hole and create a new hole on the bottom level since, once again, 31 is too large. Finally, we are able to place 31 in the hole (Fig. 6.11). This general strategy is known as a **percolate down.** We use the same technique as in the `insert` routine to avoid the use of swaps in this routine.

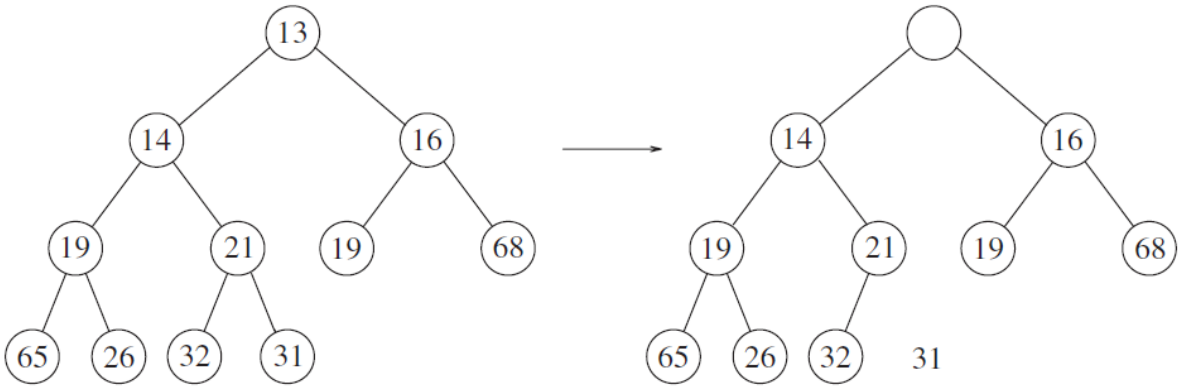

Figure 6.9 Creation of the hole at the root

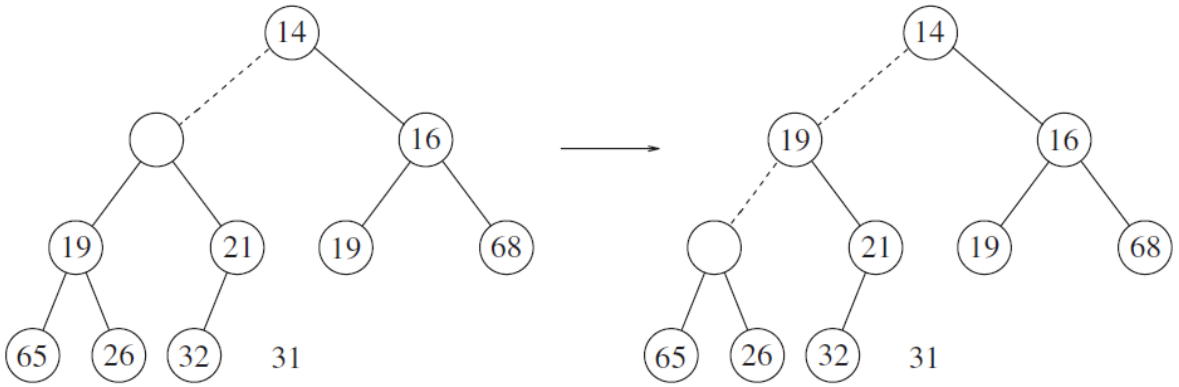

Figure 6.10 Next two steps in `deleteMin`

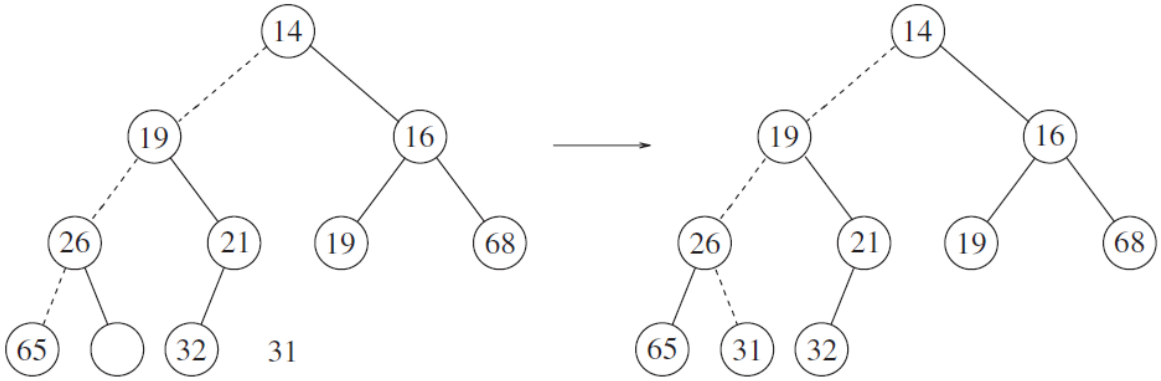

Figure 6.11 Last two steps in `deleteMin`